

Процесс сбора данных и загрузки в PostgreSQL

1. Сбор данных

Используются данные из OpenStreetMap (OSM), скаченные в формате shape-файлов с сайта Geofabrik. Для нашей БД нужно собрать данные для следующих таблиц:

- ⑩ nodes и edges — для построения графа маршрутов.
- ⑩ place_category и node_category — для классификации точек.
- ⑩ landmarks — для достопримечательностей.
- ⑩ users, routes, route_nodes — эти данные создаются в приложении.

Скачивание данных с Geofabrik

1. Перейдите на сайт Geofabrik.
2. Выберем регион, который нас интересует. (Санкт-Петербург)
3. Скачиваем shape-файлы, которые проще для обработки. Распаковываем zip-архив. Находим файл gis_osm_roads_free_1.shp (для дорог) и gis_osm_pois_free_1.shp (для точек интереса, таких как достопримечательности).

2. Подготовка данных

Shape-файлы содержат геометрические данные (линии дорог и точки интереса), которые нужно преобразовать в формат, подходящий для нашей БД. Мы будем:

- ⑩ Извлекать узлы (nodes) и рёбра (edges) из дорог.
- ⑩ Извлекать категории (place_category) и связывать их с узлами (node_category).
- ⑩ Извлекать достопримечательности (landmarks) из точек интереса.

2.1. Установка необходимых инструментов

Для работы с shape-файлами и их загрузки в PostgreSQL вам понадобятся:

ogr2ogr (часть GDAL) — для конвертации shape-файлов в PostgreSQL.

Установить GDAL: `sudo apt update sudo apt install -y gdal-bin`

2.2. Очистка существующей БД

Поскольку ваша БД уже содержит тестовые данные, сначала очистим её, чтобы избежать дубликатов:

```
-- Удаляем данные из таблиц (с учётом зависимостей)
DELETE FROM route_nodes;
DELETE FROM routes;
DELETE FROM landmarks;
DELETE FROM node_category;
DELETE FROM place_category;
DELETE FROM edges;
DELETE FROM nodes;
DELETE FROM users;
-- Сбрасываем последовательности (SERIAL)
ALTER SEQUENCE edges_edge_id_seq RESTART WITH 1;
ALTER SEQUENCE routes_route_id_seq RESTART WITH 1;
```

2.3.1 Загрузка shape-файлов в PostgreSQL

Мы загрузим два shape-файла:

- ⑩ gis_osm_roads_free_1.shp — для дорог (узлы и рёбра).
- ⑩ gis_osm_pois_free_1.shp — для точек интереса (достопримечательности).

1. **Загрузка дорог:** надо использовать ogr2ogr для загрузки shape-файла в PostgreSQL:

```
ogr2ogr -f "PostgreSQL" PG:"host=localhost user=youruser dbname=yourdb
password=yourpassword" gis_osm_roads_free_1.shp -nln temp_roads -lco
GEOMETRY_NAME=geom -lco FID=gid

youruser = postgres
yourdb = PolyWay
yourpassword = qwerty
```

Параметр -nln temp_roads создаёт временную таблицу temp_roads.

Параметр -lco GEOMETRY_NAME=geom указывает имя геометрического столбца.

Параметр -lco FID=gid задаёт имя столбца для идентификатора.

После выполнения этой команды в вашей БД появится таблица temp_roads с полями, такими как:

- ⑩ gid (идентификатор строки).
- ⑩ geom (геометрия, обычно MULTILINESTRING или LINESTRING).
- ⑩ fclass (тип дороги, например, motorway, residential).
- ⑩ name (название дороги).
- ⑩ maxspeed, oneway и другие поля.

2.3.2 **Загрузка точек интереса:** Аналогично загрузим точки интереса:

```
ogr2ogr -f "PostgreSQL" PG:"host=localhost user=youruser dbname=yourdb
password=yourpassword" gis_osm_pois_free_1.shp -nln temp_roads -lco
GEOMETRY_NAME=geom -lco FID=gid

youruser = postgres
yourdb = PolyWay
yourpassword = qwerty
```

Это создаст таблицу temp_pois с полями, такими как:

- ⑩ gid (идентификатор).
- ⑩ geom (геометрия, обычно POINT).
- ⑩ fclass (тип точки, например, monument, museum).
- ⑩ name (название точки).

2.4. Преобразование данных для вашей схемы

Теперь преобразуем данные из temp_roads и temp_pois в формат, подходящий для ваших таблиц.

2.4.1. Извлечение узлов (nodes) из дорог

Узлы должны извлекаться как начальные/конечные точки линий и точки пересечений (методы processJunctions и processRoads). Мы сделаем это с помощью PostGIS:

⑩ Извлечение начальных и конечных точек:

```
-- Создаём временную таблицу для узлов
CREATE TABLE temp_nodes (
    node_id SERIAL PRIMARY KEY,
    geom GEOMETRY(POINT, 4326)
);
-- Извлекаем начальные и конечные точки линий
INSERT INTO temp_nodes (geom)
SELECT ST_StartPoint(geom) AS geom
FROM temp_roads
WHERE ST_GeometryType(geom) IN ('ST_LineString', 'ST_MultiLineString')
UNION
SELECT ST_EndPoint(geom) AS geom
FROM temp_roads
WHERE ST_GeometryType(geom) IN ('ST_LineString', 'ST_MultiLineString');
```

⑩ Извлечение точек пересечений:

```
INSERT INTO temp_nodes (geom)
SELECT DISTINCT unnested.geom
FROM temp_roads r1
JOIN temp_roads r2 ON r1.gid != r2.gid
CROSS JOIN LATERAL ST_DumpPoints(ST_Intersection(r1.geom, r2.geom)) AS
unnested
WHERE ST_Intersects(r1.geom, r2.geom);
```

⑩ Удаление дубликатов и заполнение таблицы nodes: (через создание последовательности вручную)

```
CREATE SEQUENCE nodes_node_id_seq
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1;

ALTER TABLE nodes
    ALTER COLUMN node_id
    SET DEFAULT nextval('nodes_node_id_seq');

SELECT setval('nodes_node_id_seq', (SELECT COALESCE(MAX(node_id), 0) + 1
FROM nodes));

INSERT INTO nodes (node_id, name, latitude, longitude, type, geom)
SELECT DISTINCT ON (ST_SnapToGrid(geom, 0.00001))
    nextval('nodes_node_id_seq') AS node_id,
    NULL AS name, -- Имя можно добавить позже
    ST_Y(geom) AS latitude,
    ST_X(geom) AS longitude,
    'waypoint' AS type,
```

```
geom
FROM temp_nodes;
```

2.4.2. Создание рёбер (edges)

Теперь создаём рёбра, соединяя узлы вдоль линий:

```
-- Временная таблица для рёбер
CREATE TABLE temp_edges (
    start_node_id BIGINT,
    end_node_id BIGINT,
    distance NUMERIC
);

-- Для каждой линии находим ближайшие узлы к её началу и концу
INSERT INTO temp_edges (start_node_id, end_node_id, distance)
SELECT
    (SELECT node_id FROM nodes n WHERE ST_DWithin(n.geom,
    ST_StartPoint(r.geom), 0.00001) LIMIT 1) AS start_node_id,
    (SELECT node_id FROM nodes n WHERE ST_DWithin(n.geom,
    ST_EndPoint(r.geom), 0.00001) LIMIT 1) AS end_node_id,
    ST_Length(r.geom::geography) AS distance
FROM temp_roads r
WHERE ST_GeometryType(geom) IN ('ST_LineString', 'ST_MultiLineString')
AND (SELECT node_id FROM nodes n WHERE ST_DWithin(n.geom,
    ST_StartPoint(r.geom), 0.00001) LIMIT 1) IS NOT NULL
AND (SELECT node_id FROM nodes n WHERE ST_DWithin(n.geom,
    ST_EndPoint(r.geom), 0.00001) LIMIT 1) IS NOT NULL;

-- Заполняем таблицу edges
INSERT INTO edges (start_node_id, end_node_id, distance)
SELECT start_node_id, end_node_id, distance
FROM temp_edges
WHERE start_node_id != end_node_id; -- Исключаем петли
```

2.4.3. Извлечение категорий (place_category и node_category)

В shape-файле temp_roads есть поле fclass (тип дороги). Используем его для создания категорий:

```
-- Заполняем place_category
INSERT INTO place_category (category_id, name)
SELECT row_number() OVER () AS category_id, fclass
FROM (SELECT DISTINCT fclass FROM temp_roads WHERE fclass IS NOT NULL)
AS categories;

-- Связываем узлы с категориями через node_category
INSERT INTO node_category (node_id, category_id)
SELECT DISTINCT n.node_id, pc.category_id
FROM nodes n
JOIN temp_roads r ON ST_DWithin(n.geom, r.geom, 0.00001)
JOIN place_category pc ON r.fclass = pc.name;
```

2.4.4. Извлечение достопримечательностей (landmarks)

Используем таблицу temp_pois для извлечения достопримечательностей:

```
-- Добавляем точки интереса в nodes
```

```

INSERT INTO nodes (node_id, name, latitude, longitude, type, geom)
SELECT
    nextval('nodes_node_id_seq') AS node_id,
    name,
    ST_Y(geom) AS latitude,
    ST_X(geom) AS longitude,
    'landmark' AS type,
    geom
FROM temp_pois
WHERE fclass IN ('monument', 'museum', 'castle', 'tourist_attraction');

-- Заполняем landmarks
INSERT INTO landmarks (landmark_id, node_id, short_description,
long_description, audio_file)
SELECT
    nextval('landmarks_landmark_id_seq') AS landmark_id,
    n.node_id,
    t.name AS short_description,
    t.name || ' - историческое место' AS long_description,
    NULL AS audio_file -- Можно добавить путь к аудиофайлу позже
FROM temp_pois t
JOIN nodes n ON ST_DWithin(n.geom, t.geom, 0.00001)
WHERE t.fclass IN ('monument', 'museum', 'castle',
'tourist_attraction');

```

2.5. Очистка временных таблиц

После загрузки данных удалим временные таблицы:

```

DROP TABLE temp_roads;
DROP TABLE temp_pois;
DROP TABLE temp_nodes;
DROP TABLE temp_edges;

```

3. Проверка данных

Убеждаемся, что данные корректно загружены:

```

SELECT * FROM nodes LIMIT 10;
SELECT * FROM edges LIMIT 10;
SELECT * FROM place_category LIMIT 10;
SELECT * FROM node_category LIMIT 10;
SELECT * FROM landmarks LIMIT 10;
SELECT * FROM users;
SELECT * FROM routes;
SELECT * FROM route_nodes;

```

	route_id [PK] bigint	node_id [PK] bigint	node_order integer
1	1	1078508	1
2	1	1649292	2
3	1	2649203	3